

LEOLANI

A Knowledge-Graph-Driven Chat Agent

Grounding a lifestyle-coaching conversation in a living knowledge graph — architecture, conversational strategies, and a live example.

Piek Vossen — Vrije Universiteit Amsterdam

ctrl-kg-driven-chat

Type 2 diabetes self-management

WHY THIS AGENT EXISTS

A coach that remembers, not just a chatbot that replies

Structured diary construction

Every turn is turned into semantic-role triples and pushed into a knowledge graph — a persistent, queryable record of what the person actually did and felt.

The graph drives the dialogue

What's still missing in the graph — not a fixed script — decides the agent's next question, so the conversation stays grounded in this person's own history.

Built for Type 2 diabetes

Diet, exercise, sleep, mood, medication and symptoms — everyday factors that shape blood-sugar management, tracked over time, not just one session.

Building on structured-diary and synthetic-dialogue work for functioning & well-being monitoring — see References.

HIGH-LEVEL ARCHITECTURE

Two pipelines, one shared graph



Repeats every turn — the graph accumulates across the whole conversation, not just one exchange.

chat_sessions.py

Orchestrates every turn — ChatSession → KgChatSession → KgIntentChatSession

kg_chat_gui.py

Chat window plus a live node-link graph panel

catch_up_from_kg.py

The saturation loop that opens and paces a session

From a sentence to graph triples

JAN SAYS

"I cycled for an hour yesterday"

agent_patient	Jan
time	yesterday
qualification	for an hour

- **Incremental, not batch** — one LLM call per turn keeps the running history, so "it" and "that" still resolve to the right earlier activity.
- **One id per real-world event** — the same *activity_id* is reused every time it's mentioned again, even with no new phrase.
- **Pronouns become identities** — "I" and "you" link to a real person's URI, not a bare, unlinked word.
- **One shared role vocabulary** — agent, patient, location, time, qualification — under the project's own *n2mu:* namespace.

GROUNDED IN REAL CONVERSATIONAL DATA

256 synthetic conversations, one lifestyle coach

256

conversations

2,003

annotated turns

2,656

unique activities

3,506

output entries

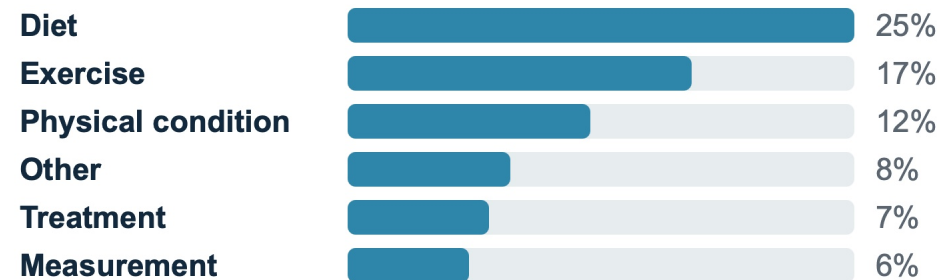
1,047

time expressions —
842 resolved

What's annotated, per turn

- Activity and condition mentions, typed (diet, exercise, treatment, symptom ...) with a stable id reused across turns
- Semantic roles — agent, patient, agent_patient, experiencer, instrument, result, location, time
- Time expressions resolved to point, range, duration, recurring or vague
- Speaker perspective per turn — emotion, factuality, certainty

Top activity types, by share



ChatGPT-generated chats across 16 fake diabetes patients (age, gender, ethnicity, biography), randomly grounded January 2010 – June 2015 (Ntanavaras, de Boer & Vossen, 2026 — see References).

An event-centric knowledge graph, not a transcript

JAN SAYS

"I'm concerned about my frequent thirst and dry mouth."

thirst_and_dry_mouth_1

instanceOf	EventSeries
experiencer	Jan
time	frequent
attributedTo → emotion	fear

- Every mention is an **EventSeries** instance — not a flat triple. The same identifier is reused every time thirst comes up again, exactly like *activity_id* in this agent's own graph.
- Perspective travels with the event** — *attributedTo* links it to an attribution node carrying the speaker's emotion, factuality and certainty. A memory, not just a fact.
- One shared role set** — agent, patient, agent_patient, experiencer, instrument, location, time, result — the same vocabulary this agent's own n2mu roles build on.

Distinct entities extracted from the corpus, by type

432	359	274	221	216	62	58
activities	objects	instruments	properties	time expr.	agents	places

Two ways to spot a knowledge gap

KG_GAP_FINDER.PY

Peer statistics

Compares an activity to similar past instances. A gap fires once a majority of peers share the missing predicate.

Limit: the very first instance of a kind has no peers yet — no gap can ever fire for it.

INTENT_GAP_FINDER.PY

Hand-authored intents

One JSON file per topic — diet, exercise, sleep, symptoms, medication, conditions, measurements — each stating what a complete record needs.

Works from the very first mention — the expectation is declared, not inferred from peers.

Same per-turn loop either way — KgChatSession vs. KgIntentChatSession swap only the gap-finding step.

Ask one thing at a time, in priority order



measurement_intents.json, in practice

"Did you measure *blood sugar level* lately?" → "What did you measure?" (quantity) → "When did you measure it?" — each question only paraphrases the intent's own hand-written template, so a weaker model never leaks a raw role name like "patient" into the question.

Capture the answer, don't lose it to a coreference miss



Confirm, don't interrogate

A missing "who" is asked as "Was that you?" — not an open question. Yes / no / a correction is pushed straight to the graph.



Attach the reply directly

A single-predicate gap arms a pending slot. The next reply is classified ANSWER / DECLINE / UNRELATED and written straight onto that same activity — never left for the extractor to re-link.

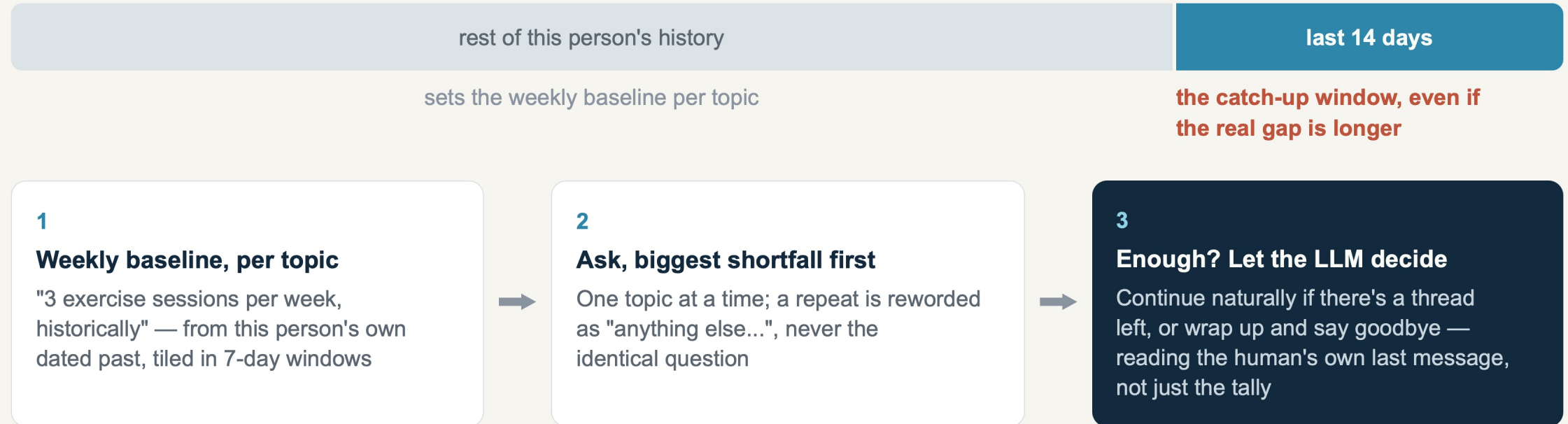


Never ask forever

Two backstops: a gap already asked is never repeated, and a requirement gets a fixed number of tries across the whole chat before the agent moves on.

Why it matters: a short reply like "30 minutes" often gets coreferenced onto a brand-new, empty subject — direct capture skips that guesswork entirely.

Ask until there's enough — not just once



Ask "what did we discuss?" any time — a plain recap of what's covered vs. still open, with no effect on the loop itself.

An acknowledgement is never the whole reply

BEFORE

"Thanks for letting me know."

The reply, and the conversation, just stop there.

AFTER

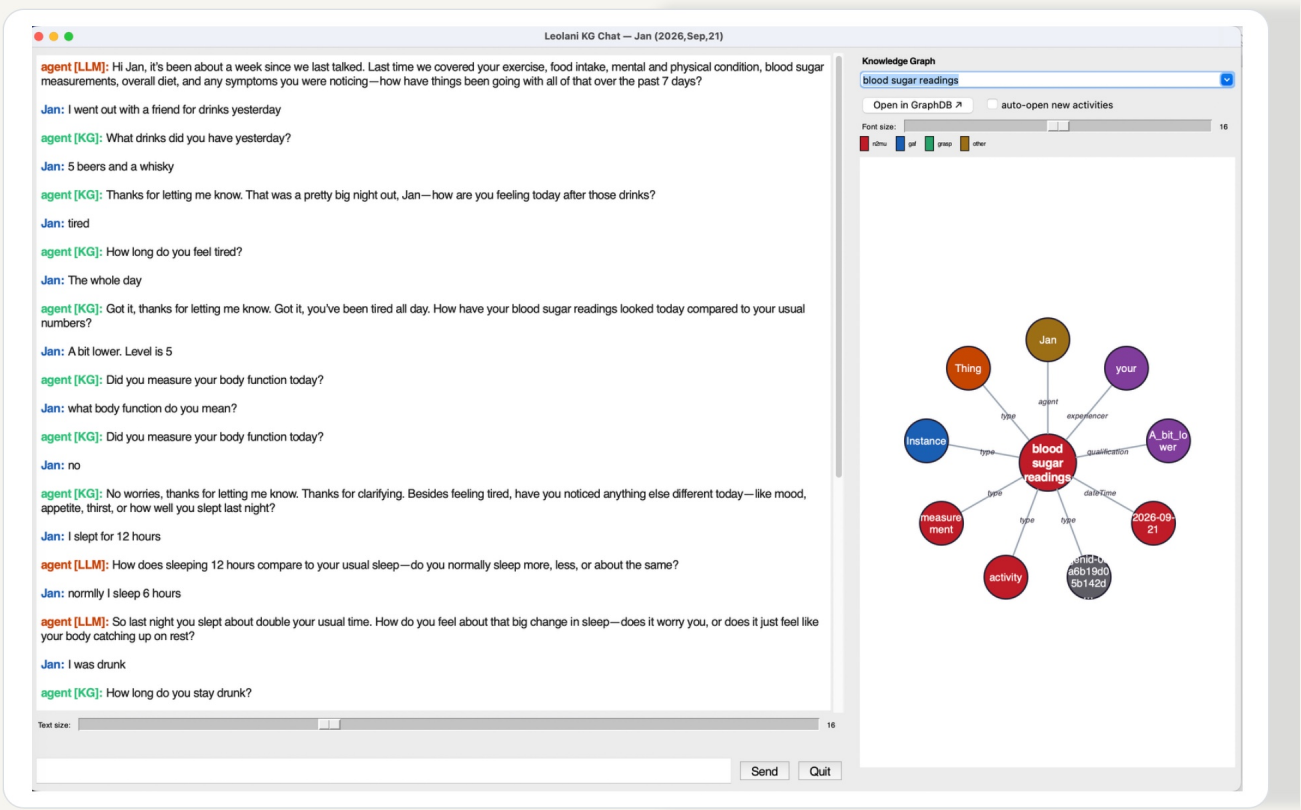
"No worries, thanks for letting me know. How has your exercise been going lately?"

Same acknowledgement — immediately chained into the next open topic.

Every acknowledgement now calls straight into the agent's own reply function afterward — the exact call the conversation would make anyway once nothing else is pending, made explicit so it always fires.

A LIVE EXAMPLE — OVERVIEW

Catching up with Jan



Opens with a recap

Names last time's topics, then asks one specific question to start

Follow-ups drill down

Drinks, duration, blood sugar — one intent question at a time

The graph, live

Every fact lands on the right — here, "blood sugar readings" grounded by agent, qualification and date

The screenshot is hard to read at a distance — full, readable transcript on the next slide →

Same conversation, read in full

LL the general reply — opening recap, pacing, or
M plain conversation

K a knowledge-graph gap — one specific missing fact about what was
G just reported

LLM "It's been about a week — last time we covered exercise, food, mood, blood sugar and more. How have things been this past week?"

Jan "I went out with a friend for drinks yesterday"

KG "What drinks did you have yesterday?"

Jan "5 beers and a whisky"

KG "Thanks for letting me know — how are you feeling today after those drinks?"

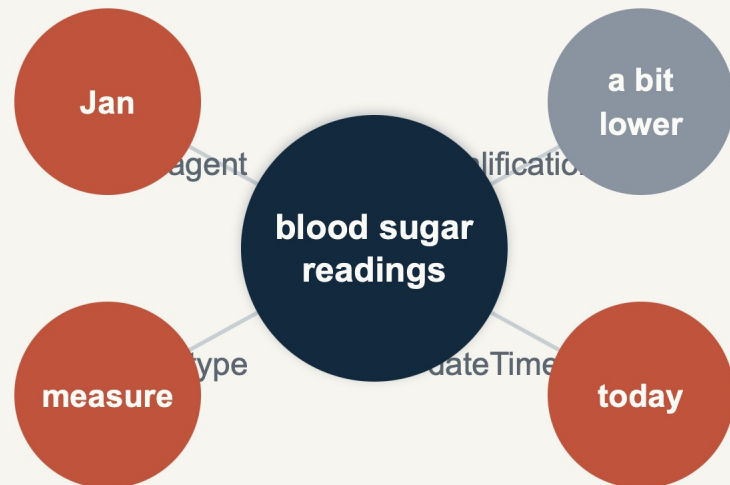
Jan "tired"

KG "How long do you feel tired?"

Jan "The whole day"

KG turns chain directly off what Jan just said, one missing fact at a time; the LLM turn paces the whole session against the graph's own history.

A live graph panel, next to the chat



Colored by namespace

Red for the project's own n2mu roles, blue/green for the shared GAF and GRASP ontologies, gold for anything else

No provenance clutter

The label predicate and pure extraction-provenance links are never drawn as their own nodes

Duplicate edges merged

An inferred sem:hasActor copy collapses onto the real n2mu:agent edge that already points at the same object

TAKEAWAYS

Grounded, paced, and never a dead end

- The graph — not a script — decides the next question, from the very first mention of a topic.
- Sessions open by measuring the real gap against this person's own history, then ask until there's genuinely enough.
- Answers are captured directly and never asked for twice; acknowledgements always keep the conversation moving.
- Every fact is visible, live, in a graph panel colored and de-duplicated for a human to actually read.

REFERENCES

Vossen, P., Báez Santamaría, S., & Baier, T. (2024). A conversational agent for structured diary construction enabling monitoring of functioning & well-being. *HHAI 2024: Proceedings of the Third International Conference on Hybrid Human-Artificial Intelligence* (pp. 315–324). SAGE Publications.

Ntanavaras, S., de Boer, M., & Vossen, P. (2026). A synthetic conversational dataset for Type 2 diabetes management. *Proceedings of the Third Workshop on Patient-Oriented Language Processing (CL4Health) @ LREC 2026* (pp. 171–181).

Reconstructing lifestyles from conversations between Diabetes patients and a coach. *LREC 2026* (under review) — the conversational corpus and event-centric knowledge graph behind this agent's memory.